Hunting for bugs in the e-ID

An overview of the swiyu ecosystem and how to test its security

REDGUARD
SECURING YOUR ASSETS

About me

BTW, the slides are here:
https://github.com/Foo-Manroot/e-ID-packer-lab

- Miguel a.k.a. @Foo-Manroot
- Pentester for a long, long time
- Pentesting the e-ID since 2025
- Not really good at making good-looking slides (sorry!)



Our goals for today

1. Learn how the e-ID (**swiyu**) works
   a. Understand them main components and how they interact with each other
2. Get a "happy flow"
3. Be able to:
   a. Debug the Issuer
   b. Debug the Verifier
   c. Debug the app
   d. Intercept the network traffic

Agenda

1. How does the e-ID work? Standards and architecture
2. swiyu open-source components
3. How the lab should work (demo)
4. Break ☕
5. Setup Burp
6. Setup the Issuer
7. Compile the Android app
8. Issue our first credential
9. Break ☕
10. Configure the verifier
11. Verify our credential

But first… What is the e-ID?

- A.k.a **Swiyu**
- The digital counterpart to Switzerland’s ID (and many other documents)
- Developed by the BIT/OFIT/UFIT
- (almost) completely open source
- Based on open standards
  - EU's digital ID (eIDAS/EUDI) also uses some of the same standards

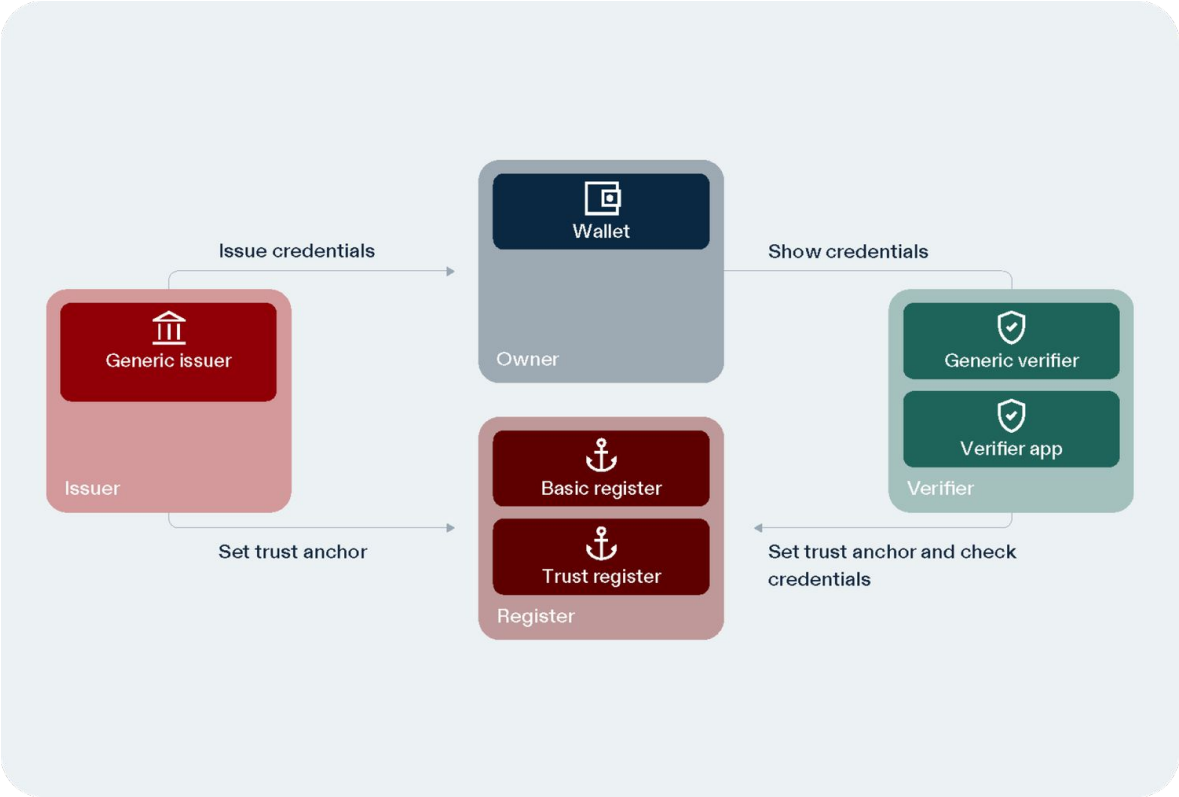


Source: https://www.eid.admin.ch/en/e-id-e

How does the e-ID work?

Standards and architecture

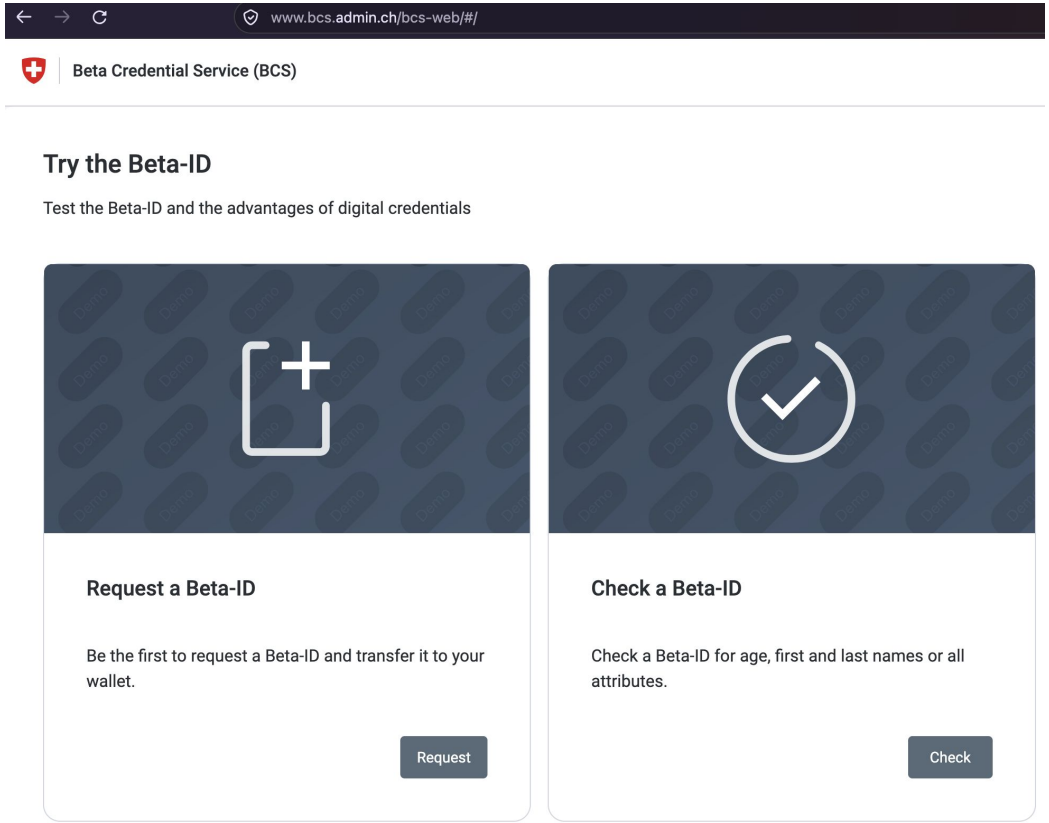
Who is who in the e-ID ecosystem



Source: https://www.eid.admin.ch/en/technology

REDGUARD

(small e2e demo)



REDGUARD

Implements a handful of standards

- OpenID for Verifiable Credential Issuance (**OID4VCI**): https://openid.net/specs/openid-4-verifiable-credential-issuance-1_0.html
- OpenID for Verifiable Presentations (**OID4VP**): https://openid.net/specs/openid-4-verifiable-presentations-1_0.html

- **DID:webvh**: https://identity.foundation/didwebvh/v1.0/
- **SD-JWT for VC**: https://datatracker.ietf.org/doc/draft-ietf-oauth-sd-jwt-vc/

- And a couple of others I'm not mentioning to avoid overcomplicating things. Head over to https://swiyu-admin-ch.github.io/specifications/ for more info

(very) Brief overview of did:webvh

- Stands for “DID Web + Verifiable History”
  - Previously known as **did:tdw** ; **swiyu used this format until very recently**

- Basically, defines how to parse, locate and verify a DID (**Decentralised Identifier**)
  - Think of it as a DNS for identities key material

- The Issuer/Verifier will say "I am *did:<whatever>:issuer.com:something*"
  - They can each have multiple DIDs

- **`did.jsonl`** aka DIDlog:. File on the server with some signatures proving the possession of a key

Hash of the first DID log entry

https://example.org:1234/some/path#key-01

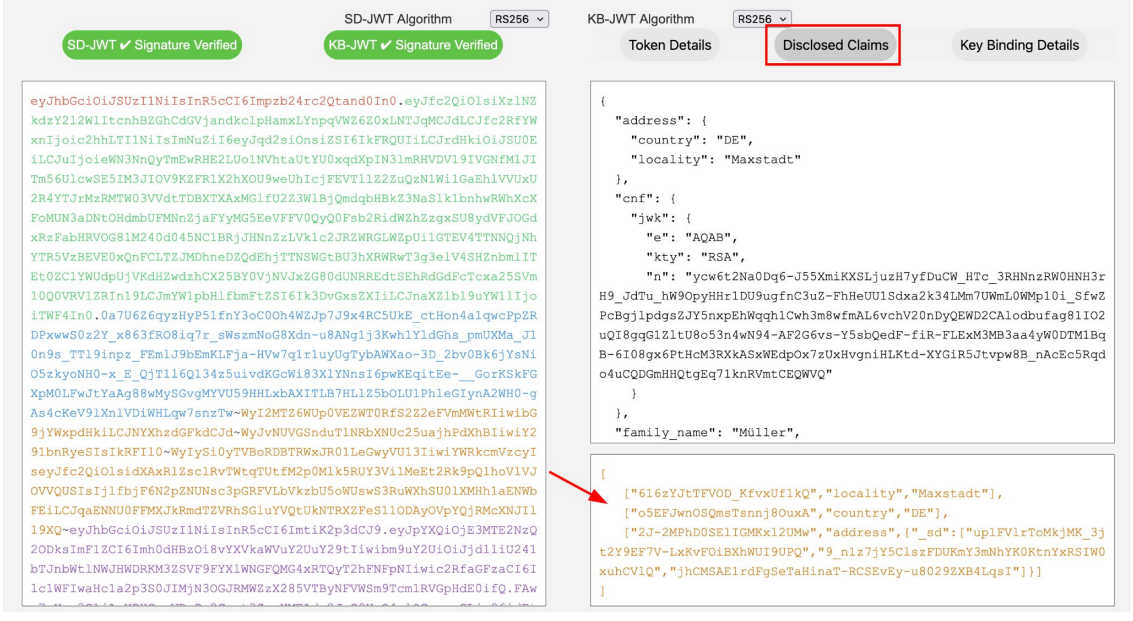
- Example:

did:webvh:QmbgyXJcQTNvDdmhqRjx2PnG3ZVD12jERMqHJ4ZSEtzTsP:**example.org%3A1234:some:path#key-01**

REDGUARD

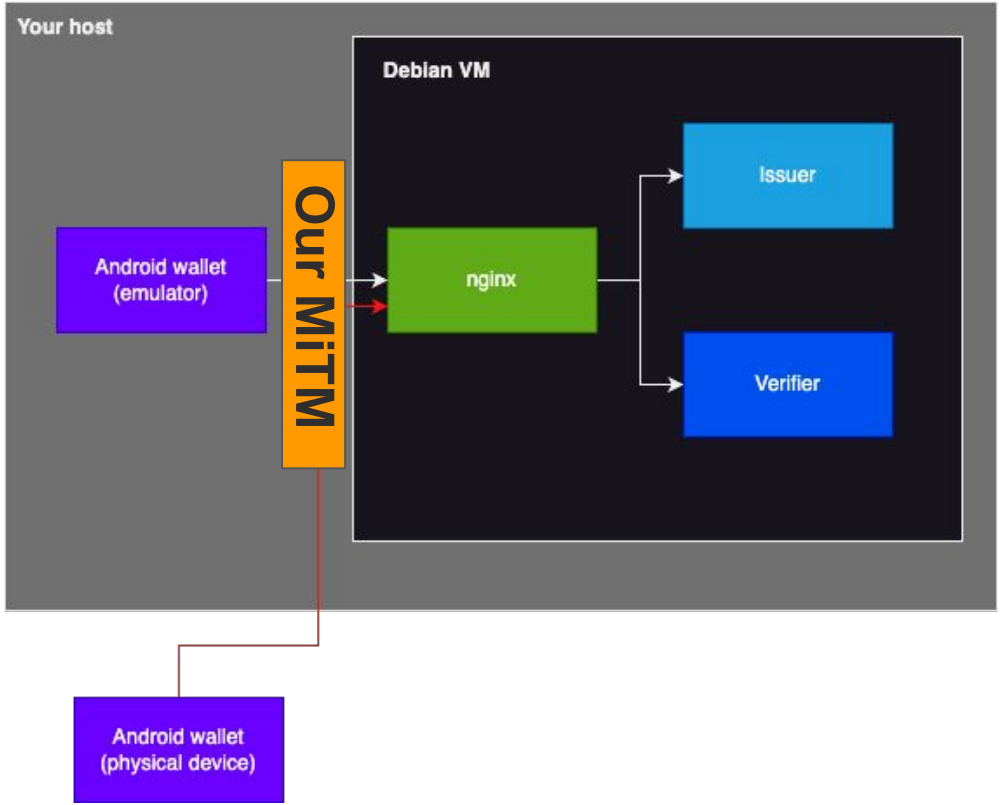
(even more) Brief overview of SD-JWT

- Selective Disclosure JWT
- Allows the holder to **disclose only selected claims** (name, age, …) while keeping a **valid signature**
- Example (from https://sdjwt.org/):



REDGUARD

How this will look in practice



swiyu open-source components

FOSS is cool

REDGUARD
SECURING YOUR ASSETS

Github is the place to be

- Everything is under the @swiyu-admin-ch organisation

- In this workshop we will use:
  - Generic Issuer: /swiyu-issuer
  - Generic Verifier: /swiyu-verifier
  - Android wallet: /eidch-android-wallet
  - DID toolbox: /didtoolbox-java

- Also, useful for testing: /swiyu-generic-test-wallet

Demo

Or: how everything should work, if everything works

Workshop procedure

1. I will show and explain the steps through and through
   - Just relax and listen, no need to stress doing the steps at the same time as me

2. If something was unclear, it’s your turn to ask

3. After the full demo you can replicate the steps
   - You can use your copy of the slides to copy+paste links, commands etc.
   - If you have questions/get stuck at some point, raise your hand
   - I will be around checking your progress and supporting you

Please, download and run the Packer script in the meantime, if you don't have the VM yet:
https://github.com/Foo-Manroot/e-ID-packer-lab



5’ break!

breath fresh air and let the new info sink in

Questions are also welcome :)

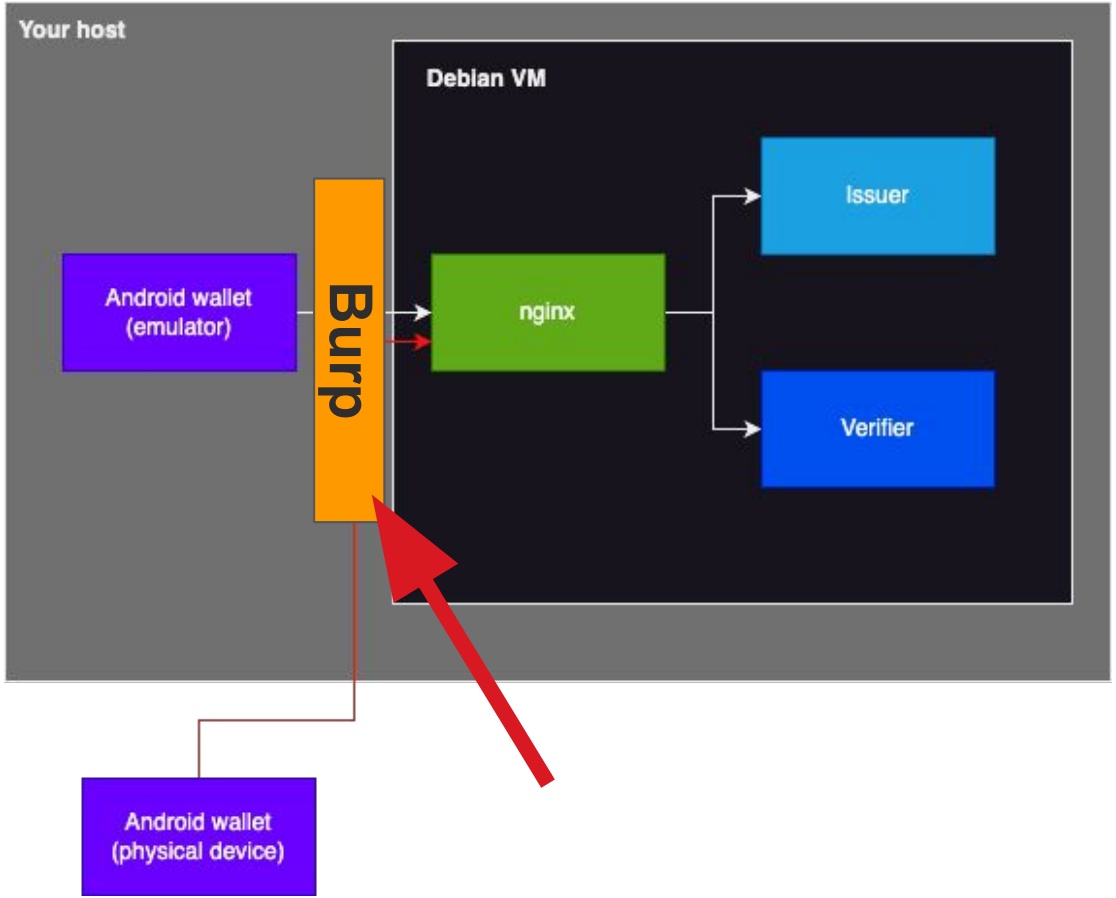
REDGUARD
SECURING YOUR ASSETS

Burp configuration

Let me see some traffic

REDGUARD
SECURING YOUR ASSETS

Reminder of our architecture



A word or two on Burp

- (HTTP) Interception proxy

- Allows you to view and edit the network traffic going through

- There are many other alternatives you may use if you prefer to:
  - Caido (https://caido.io/)
  - OWASP ZAP (https://www.zaproxy.org/)
  - mitmproxy (https://github.com/mitmproxy/mitmproxy)
  - ...

Steps

1. Download and install Burp Suite Community (https://portswigger.net/burp/releases#community)

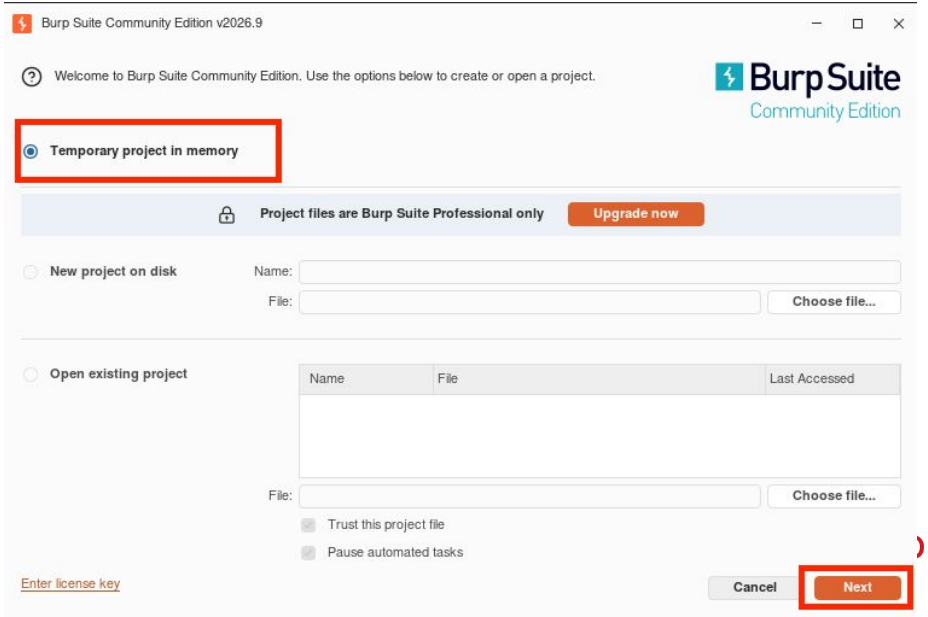
2. Start a "temporary project in memory"

3. Export Burp's CA certificate

4. Point `viscon.swiyu` to our VM

Download and install Burp

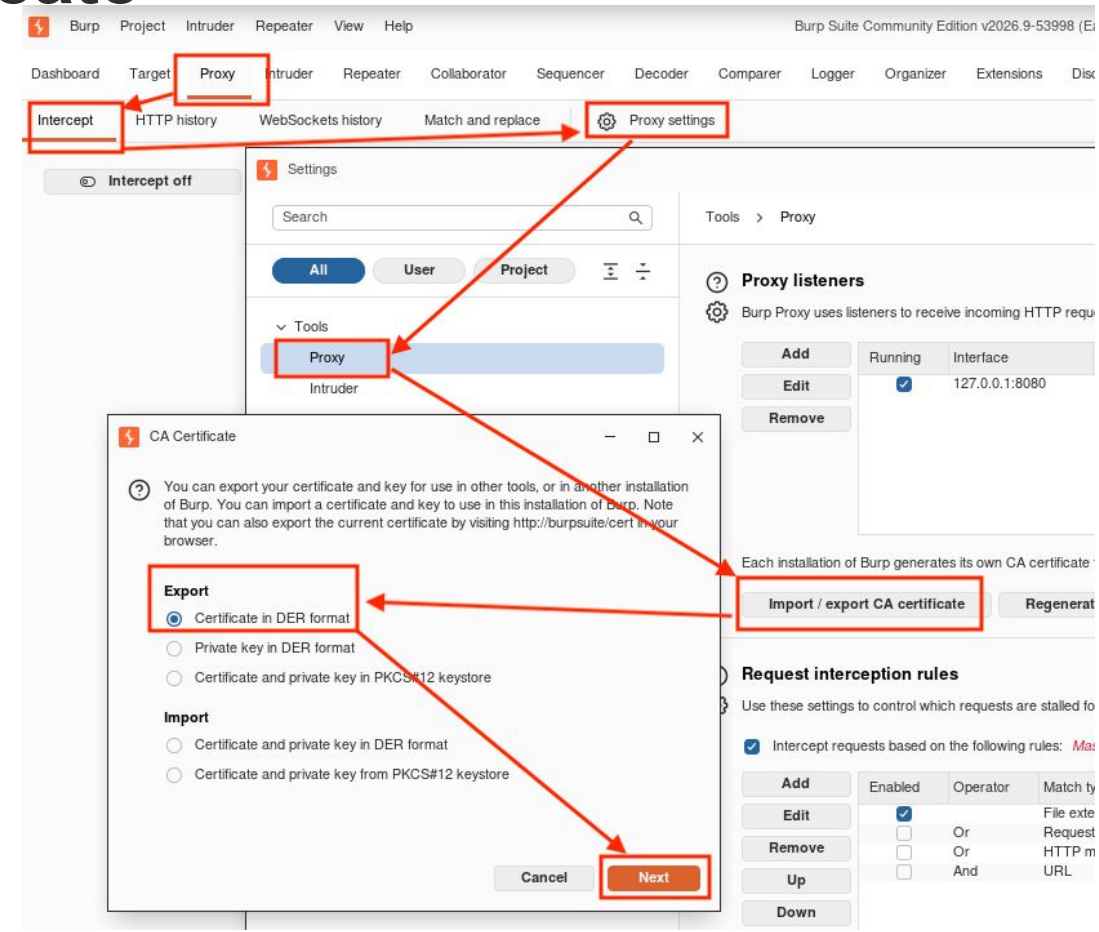
- https://portswigger.net/burp/releases#community
  - If on GNU/Linux: `bash burpsuite_linux_v2026.9.sh`
  - The defaults should be fine; just click "Next"
- When running for the first time, select "*continue with Burp Suite Community Edition*"
- "*Temporary project*"
- "*Use Burp defaults*" is fine



Export the CA certificate

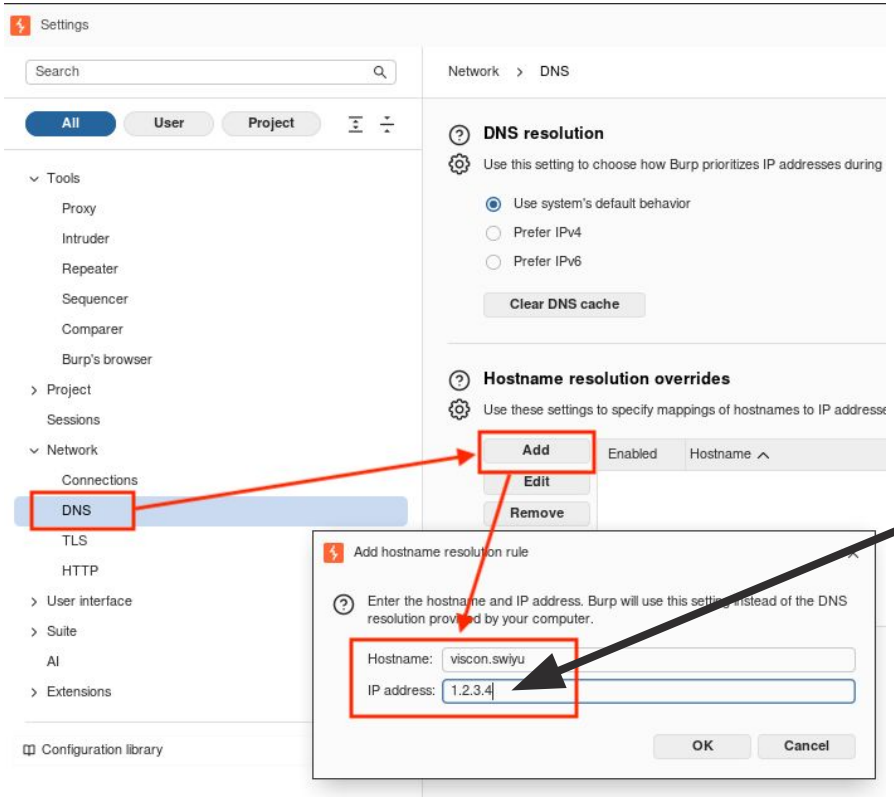
Burp automatically creates a self-signed CA certificate. We will need it to intercept the network traffic

Save it as `burp-ca.der`



DNS override

`viscon.swiyu` is not resolvable, we have to tell Burp where to find it



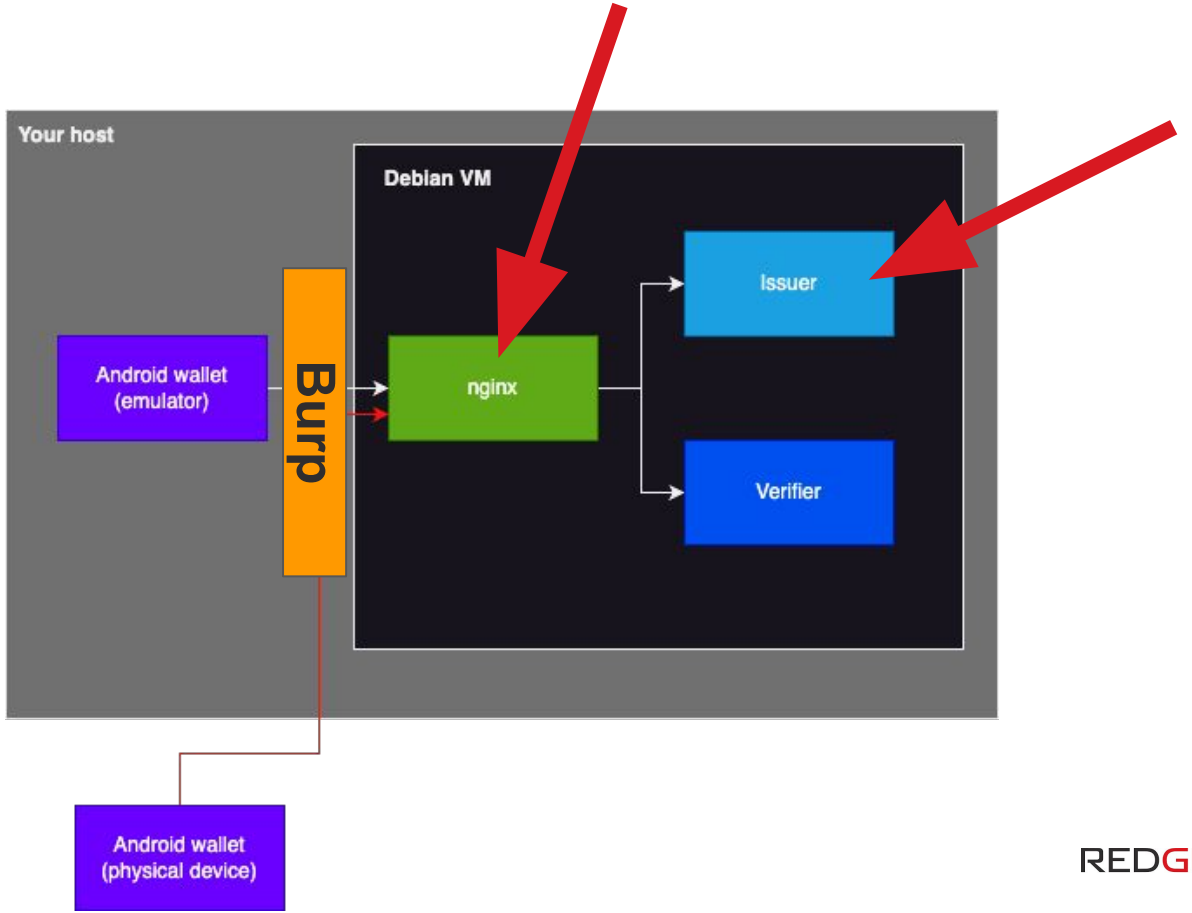
(of course, use your VM's IP address instead of 1.2.3.4)

REDGUARD

Generic issuer

Who are you, anyway?

Architecture refresher



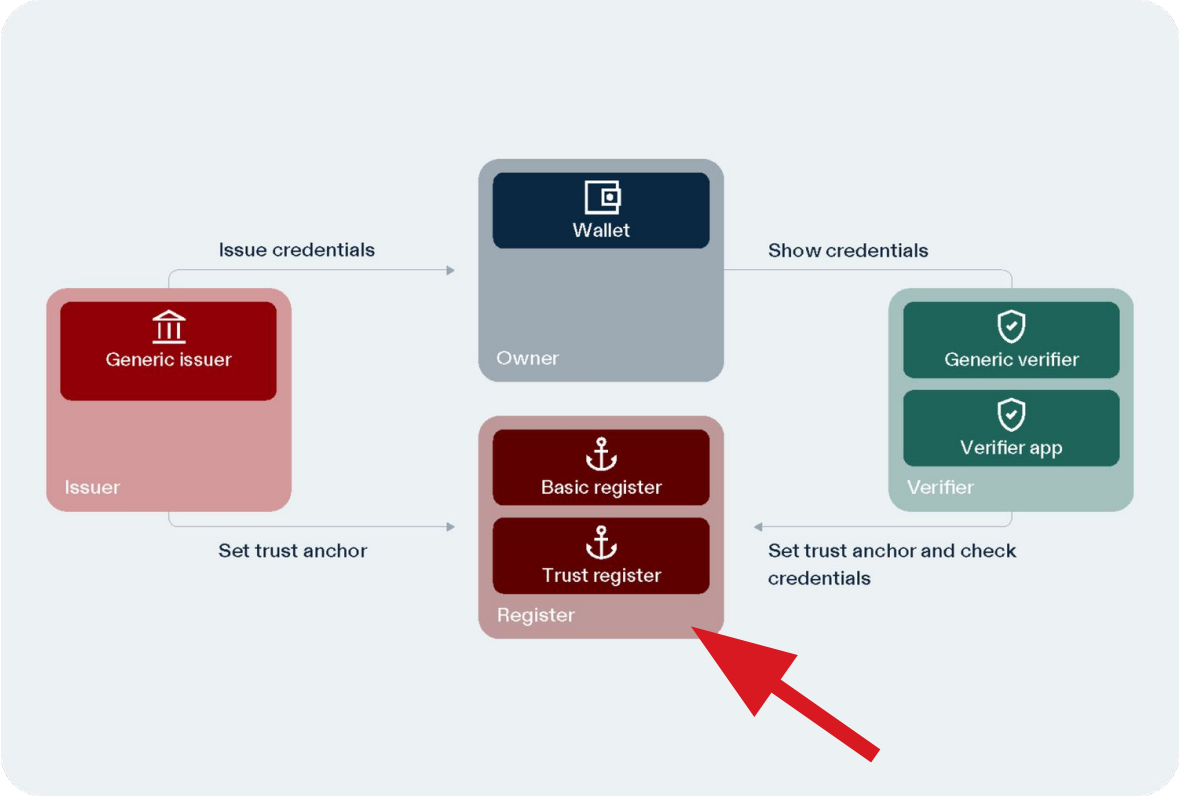
Requisites (already installed in the VM)

1. Source code: https://github.com/swiyu-admin-ch/swiyu-issuer
2. nginx (`apt install nginx`)
3. Docker (https://docs.docker.com/engine/install/debian/) + `sudo /usr/sbin/usermod -aG docker $USER` and logout+login
4. Visual Studio Code: https://code.visualstudio.com/docs/setup/linux
   - Not really a requisite, but definitely helpful
5. DID toolbox: https://github.com/swiyu-admin-ch/didtoolbox-java

Proxy (nginx)

- Simply redirects the requests to the issuer or the verifier:
  - Issuer:
    - `/issuer/`: Management endpoints. **Should be private in a real deployment**
    - `/.well-known/openid-credential-issuer`
    - `/.well-known/openid-configuration`
    - `/oid4vci/`
  - Verifier:
    - `/verifier/`: Management endpoints. **Should be private in a real deployment**
    - `/oid4vp/`
  - "Trust Registries": `/static/{issuer,verifier}/did.jsonl`

(small reminder: these are the registries)



Source: https://www.eid.admin.ch/en/technology

Some VSCode extensions in the VM

- Required:
  - `code --install-extension` **`vscjava.vscode-java-pack`**

- Just useful (e.g., to review semgrep/codeql/... results)
  - `code --install-extension trailofbits.sarif-explorer`
  - `code --install-extension trailofbits.weaudit`

DID log creation

1. Download the didtoolbox:
   https://github.com/swiyu-admin-ch/didtoolbox-java/releases/latest

2. Create a DID log:

```
$ java -jar ./target/didtoolbox-*-with-dependencies.jar create -u
'https://viscon.swiyu/static/issuer' > issuer_did.jsonl
```

3. "Upload" it to our "trust registry"

```
# cp issuer_did.jsonl /var/www/html/static/issuer/did.jsonl
```

Issuer config files

- `issuer_metadata.json`
  - Configuration of the Issuer itself: endpoints, name, supported credential types, etc.
  - Only one `issuer_metadata.json` per Issuer
  - https://swiyu-admin-ch.github.io/specifications/interoperability-profile/#credential-issuer-metadata

- `vct_metadata.json`
  - Configuration of one Verifiable Credential Type (VCT): name, resources, attributes, ...
  - One issuer can serve multiple VCTs

- `oca.json`
  - Defines how the VCT is shown (colours, translations, definitions on which attributes are "sensitive", ...)
  - Based on the OCA specs (https://oca.colossi.network/ // https://swiyu-admin-ch.github.io/specifications/oca/)
  - One per VCT

- `json_schema.json`
  - JSON schema for validation of the credential (attributes, types thereof, ...)
  - https://json-schema.org/docs
  - One per VCT

Configuring the Issuer

Our base config will be `issuer-application/src/main/resources/application-local.yml`

```
application:
  external-url: "https://viscon.swiyu/issuer"
  issuer-id: "<go to your issuer_did.jsonl and paste here the value of $.[3].value.id">

 metadata-file: "classpath:issuer_metadata.json"
  vct-metadata-files:
    my-vct-v01: "classpath:vct_metadata.json"
  overlays-capture-architecture-metadata-files:
    my-overlay-v01: "classpath:oca.json"
  json-schema-metadata-files:
    my-schema-v01: "classpath:json_schema.json"

  key:
    sdjwt: "<the key from ~/issuer_did/.didtoolbox/assert-key-01, as a string>"
    verification-method: "<$.[3].value.assertionMethod[0] from issuer_did.jsonl>"

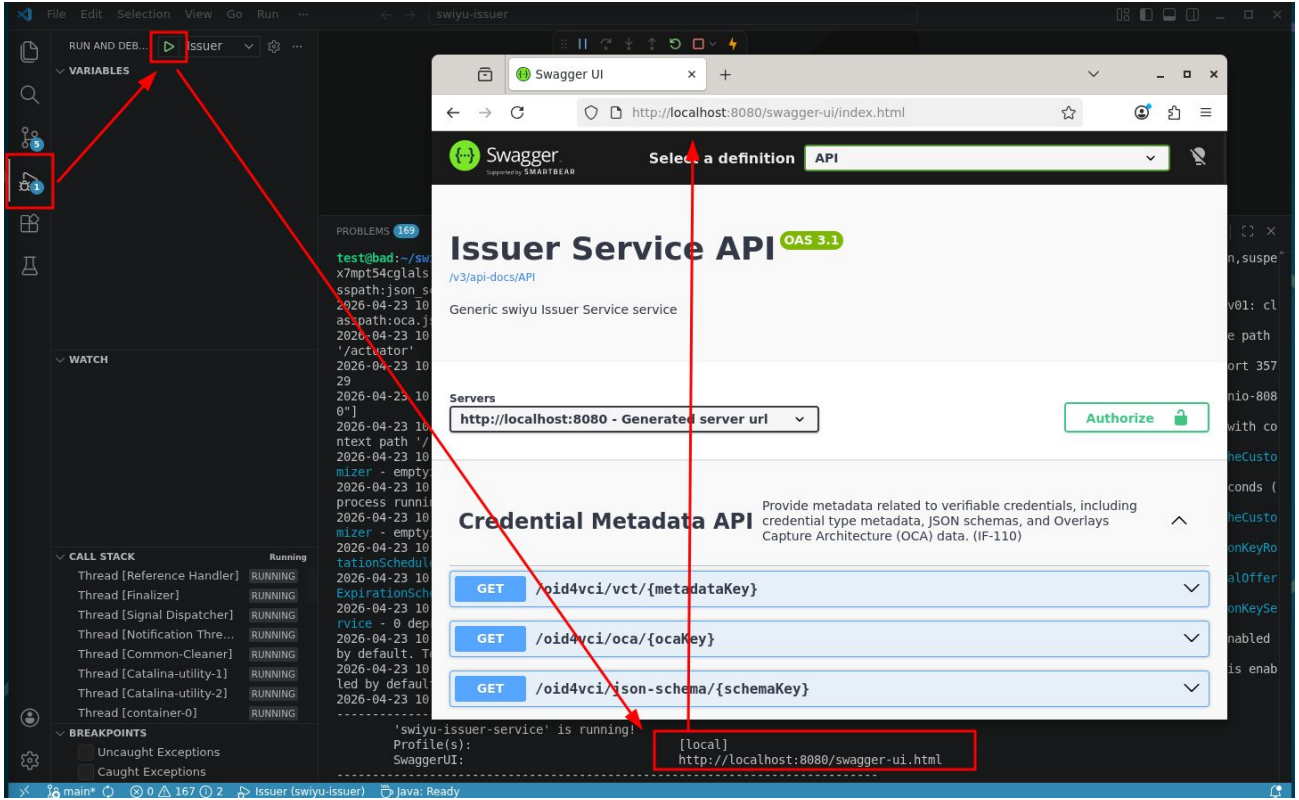
 template-replacement:
    external-url: "https://viscon.swiyu"
```

(and also remove the `.swiyu."status-registry"."token-*"` keys)

REDGUARD

Let's try it out!

`~$ code swiyu-issuer` (click "**Use Maven**" if prompted by the notification)



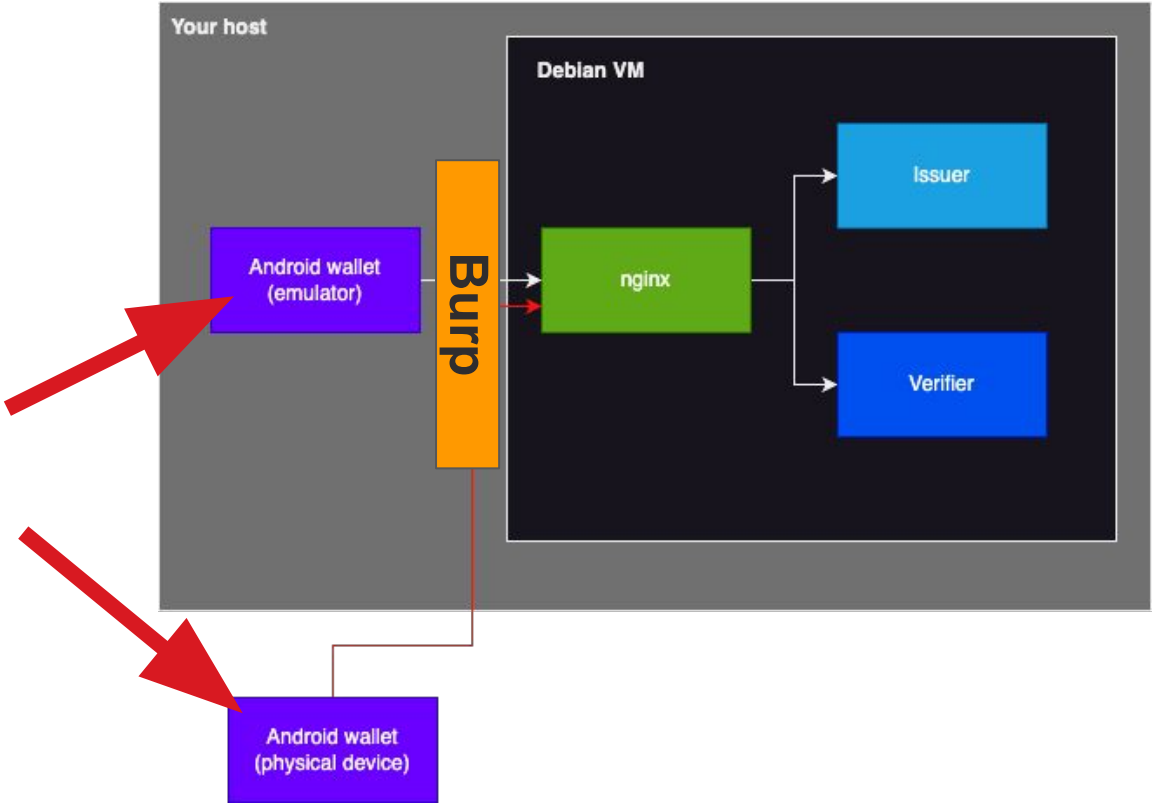
REDGUARD

Compile the swiyu wallet (Android)

Beep boop 🤖

REDGUARD
SECURING YOUR ASSETS

Architecture refresher



Requisites

1. Source code: https://github.com/e-id-admin/eidch-android-wallet

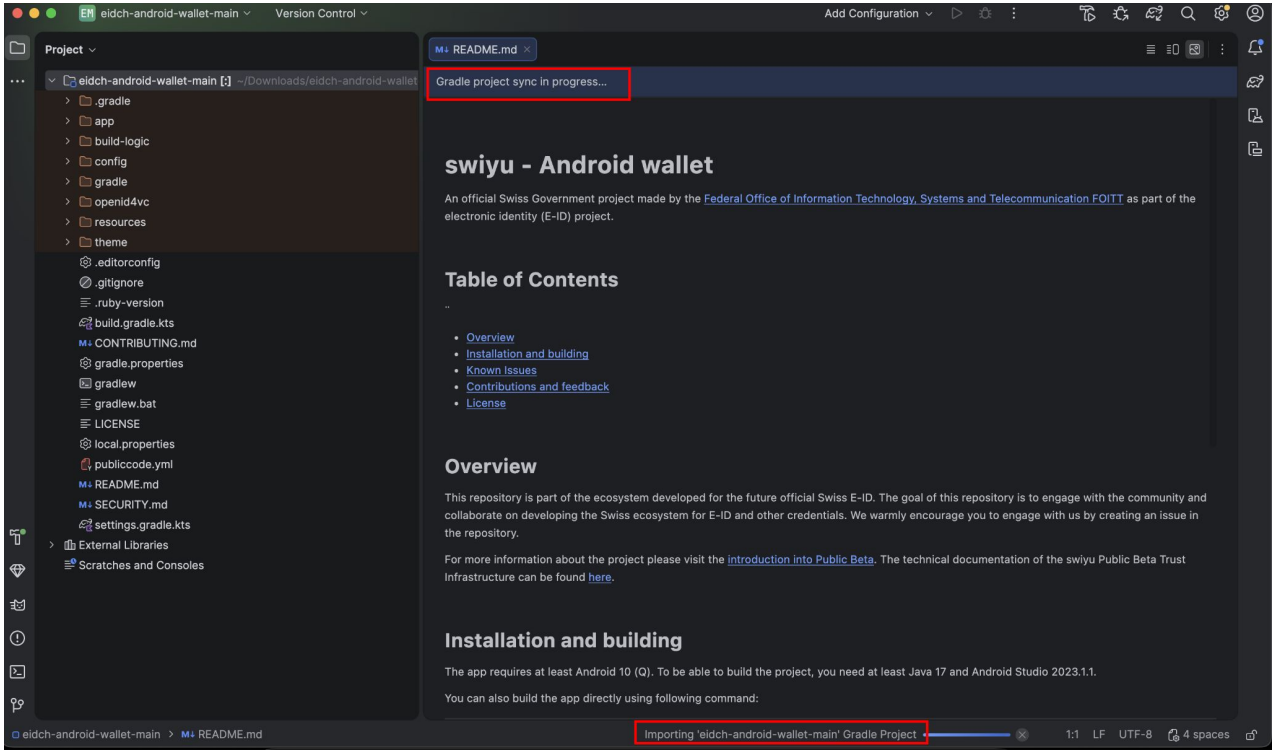
2. Android Studio: https://developer.android.com/studio

3. An Android emulated device (integrated into Android Studio)

```
$ git clone https://github.com/swiyu-admin-ch/eidch-android-wallet
```

…and then open the project with Android Studio:

(please, do it now; Android Studio takes ages to load)



Custom cert pinning

- Burp has a self-signed certificate -> we need to tell the app to accept it

1. Convert it to PEM and copy it to the app:

```
openssl x509 -in path/to/burp-ca.der -inform der -outform pem -out
./app/src/main/res/raw/root_admin_ch.pem
```

2. Override the list of pinned domains:

```
curl
https://raw.githubusercontent.com/Foo-Manroot/e-ID-packer-lab/refs/heads/main/network_
security_config.xml ./app/src/abn/res/xml/network_security_config.xml
```

Override hardcoded trusted registries

```
index 7553e3f..af0ee11 100644
--- a/app/src/main/java/ch/admin/foitt/wallet/platform/environmentSetup/data/MainEnvironmentSetupRepositoryImpl.kt
+++ b/app/src/main/java/ch/admin/foitt/wallet/platform/environmentSetup/data/MainEnvironmentSetupRepositoryImpl.kt
@@ -13,6 +13,7 @@ import javax.inject.Inject
 class MainEnvironmentSetupRepositoryImpl @Inject constructor() : EnvironmentSetupRepository {
     val prodTrustRegistryIdentifier = "identifier-reg.trust-infra.swiyu.admin.ch"
     val prodIntTrustRegistryIdentifier = "identifier-reg.trust-infra.swiyu-int.admin.ch"
+    val visconTrustRegistryIdentifier = "viscon.swiyu"

     override val userAgent: String = "swiyuWallet"

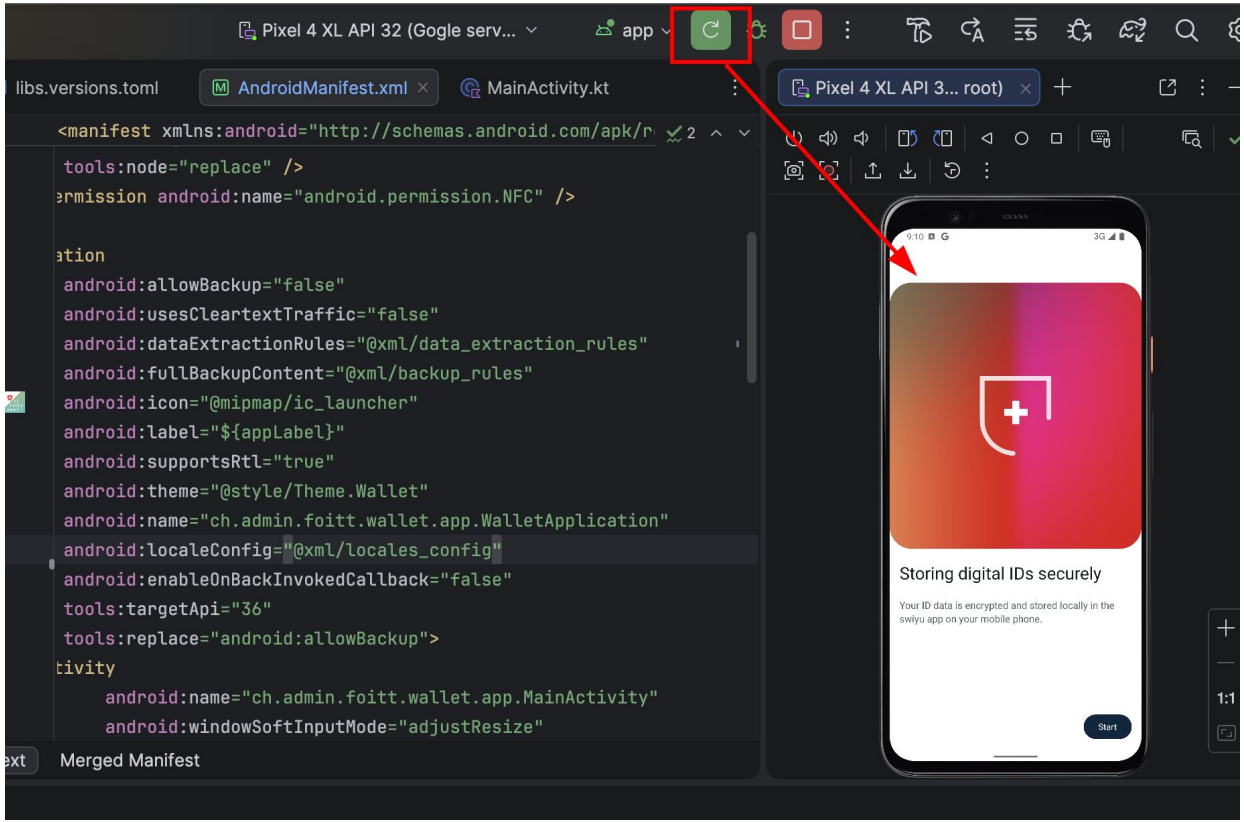
@@ -38,6 +39,7 @@ class MainEnvironmentSetupRepositoryImpl @Inject constructor() : EnvironmentSetu
     override val trustRegistryMapping: Map<String, String> = mapOf(
         prodTrustRegistryIdentifier to prodTrustRegistryUrl,
         prodIntTrustRegistryIdentifier to prodIntTrustRegistryUrl,
+        visconTrustRegistryIdentifier to prodIntTrustRegistryUrl
     )

     val prodStatusListUrl = "status-reg.trust-infra.swiyu.admin.ch"
@@ -100,7 +102,7 @@ class MainEnvironmentSetupRepositoryImpl @Inject constructor() : EnvironmentSetu
     )
     // ---------------------------- end trust protocol V2 ----------------------------

-    override val trustEnvironmentDidRegex: String = "^did:(?:tdw|webvh):[^:]+:identifier-reg\\.trust-infra\\.swiyu\\.admin\\.ch:.*"
+    override val trustEnvironmentDidRegex: String = "^did:(?:tdw|webvh):[^:]+:.*"

     @Suppress("MaximumLineLength")
     override val demoTrustEnvironmentDidRegex: String = "^did:(?:tdw|webvh):[^:]+:identifier-reg\\.trust-infra\\.swiyu-int\\.admin\\.ch:.*"
```

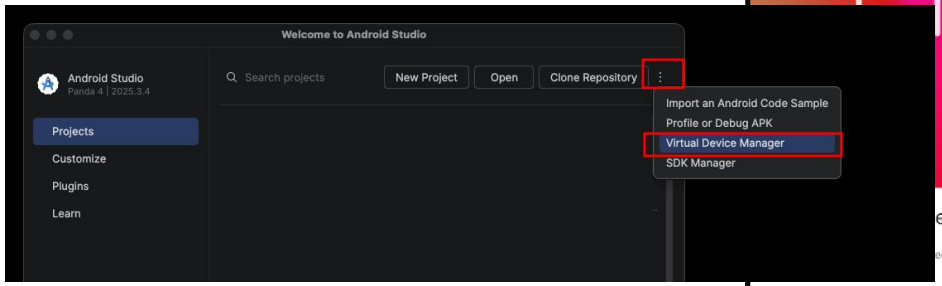
Build and run with the emulator

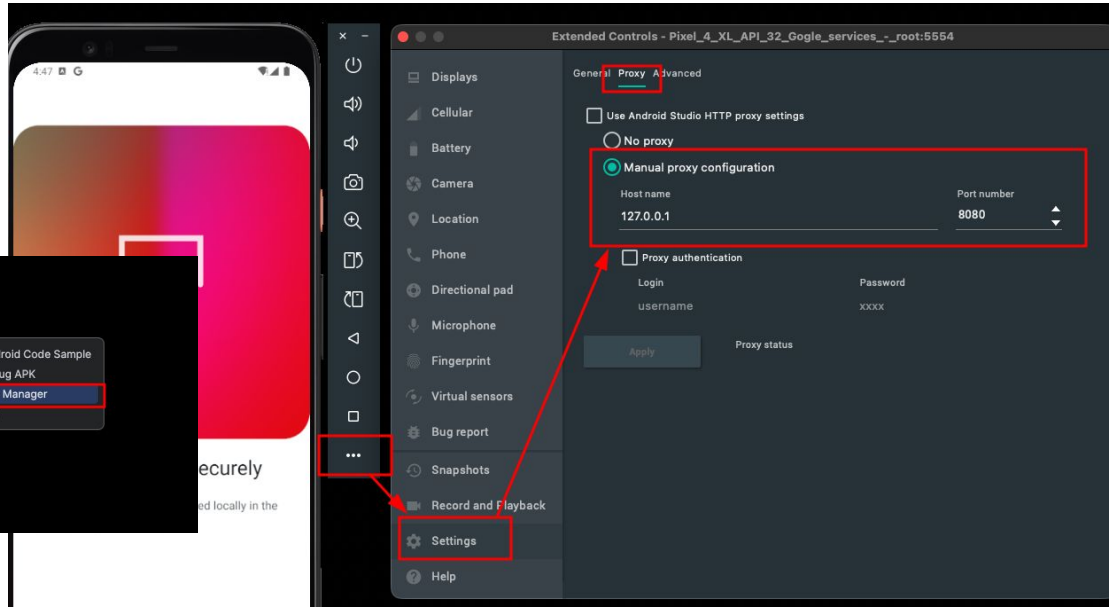


REDGUARD

Going through Burp

- If using Android **emulator**:





- If using a **physical device**:
  a. `$ adb reverse tcp:8080 tcp:8080` (on your PC)
  b. On the phone: go to "Wifi-Settings" > (click on the edit symbol) > Proxy: "manual" > hostname: "127.0.0.1" > Port: "8080" > and click "Save"

Issuing our first credential

Hi, my name is

1. Create credential offering:

```
$ curl -ki https://viscon.swiyu/issuer/management/api/credentials -H
"Content-Type: application/json" --data
'{"metadata_credential_supported_id":["visconid-sdjwt"],"credential_subjec
t_data":{"student_id":"123<h1>w</h1>ololo","given_name":"Brunets","family_
name":"Bruniwisis","age_over_16":"true","age_over_18":"false","age_over_65
":"true"},"credential_metadata":{"deferred":false},"offer_validity_seconds
":86400,"deferred_offer_validity_seconds":604800,"credential_valid_until":
"2030-01-01T19:23:24Z","credential_valid_from":"2010-01-01T18:23:24Z"}'
```


```
{"management_id":"f6...5a","offer_id":"c0...b9","offer_deeplink":"swiyu://
?credential_offer=%7B%22grants%22%3A%7B%22urn%3Aietf%3Aparams%3Aoauth%3Agr
ant-type%3Apre-authorized_code%22%3A%7B%22pre-authorized_code%22%3A%22...%
22%7D%7D%2C%22credential_issuer%22%3A%22https%3A%2F%2Fviscon.swiyu%2Fissue
r%22%2C%22credential_configuration_ids%22%3A%5B%22visconid%22%5D%7D"}
```

2. Request the credential with the swiyu wallet

Tip: `adb shell am start -a android.intent.action.VIEW -d '"swiyu://?credential_offer=...visconid%22%5D%7D"' ch.admin.foitt.swiyu.abn`

Or convert the link to a QR code (`qrencode`, Cyberchef, ...) and scan it with the swiyu App

If everything goes well, you have successfully issued your first credential!

🥳🎉🥳

15’ break!

Plz, install Android Studio and open the wallet repo if not yet done

You may also want to make sure you have the VM (via packer or downloaded from my hotspot)

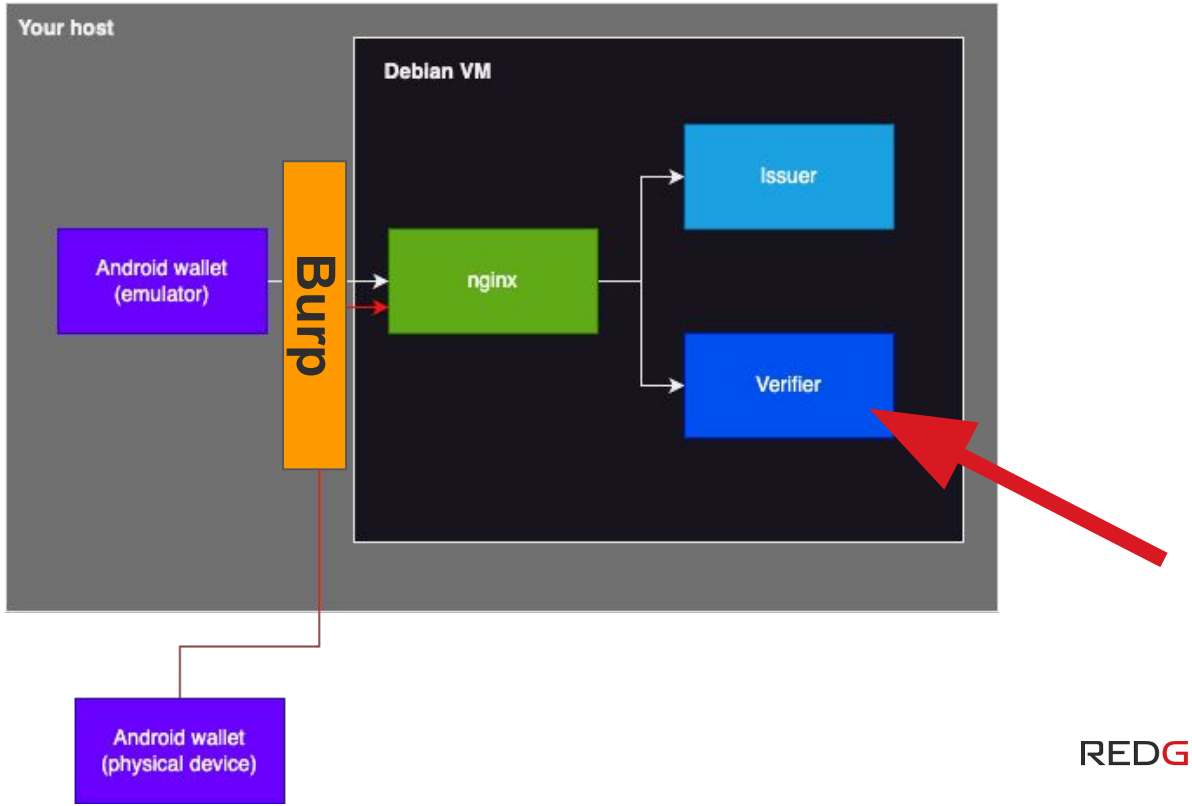
REDGUARD
SECURING YOUR ASSETS

Generic verifier

What's a credential for if we can't verify it?

REDGUARD
SECURING YOUR ASSETS

Architecture refresher



Requisites (same as the Issuer; already installed)

1. Source code: https://github.com/swiyu-admin-ch/swiyu-verifier
2. nginx (`apt install nginx`)
3. Docker (https://docs.docker.com/engine/install/debian/) + `sudo /usr/sbin/usermod -aG docker $USER` and logout+login
4. Visual Studio Code: https://code.visualstudio.com/docs/setup/linux
   - Not really a requisite, but definitely helpful
5. DID toolbox: https://github.com/swiyu-admin-ch/didtoolbox-java

DID log creation (again, same as the Issuer)

1. Download the didtoolbox:
   https://github.com/swiyu-admin-ch/didtoolbox-java/releases/latest

2. Create a DID log:

```
$ java -jar ./target/didtoolbox-*-with-dependencies.jar create -u
'https://viscon.swiyu/static/verifier' > verifier_did.jsonl
```

3. "Upload" it to our "trust registry"

```
# cp issuer_did.jsonl /var/www/html/static/ verifier/did.jsonl
```

Not much config is needed

We have to fix some config files to prevent clashes with the issuer:

- `verifier-application/compose.yaml`: change `services.postgres.ports` from `'5434:5432'` to `'5435:5432'`

- `verifier-application/src/main/resources/bootstrap.yml`: change `server.port` from `8080` to **`8081`**

Configuring the Verifier

Our base config will be `verifier-application/src/main/resources/application-local.yml`

```
application:
  signing_key: "<the key from ~/verifier_did/.didtoolbox/assert-key-01, as a string>"
  signing-key-verification-method: "<go to your verifier_did.jsonl and paste here the
value of $.[3].value.assertionMethod[0]>"

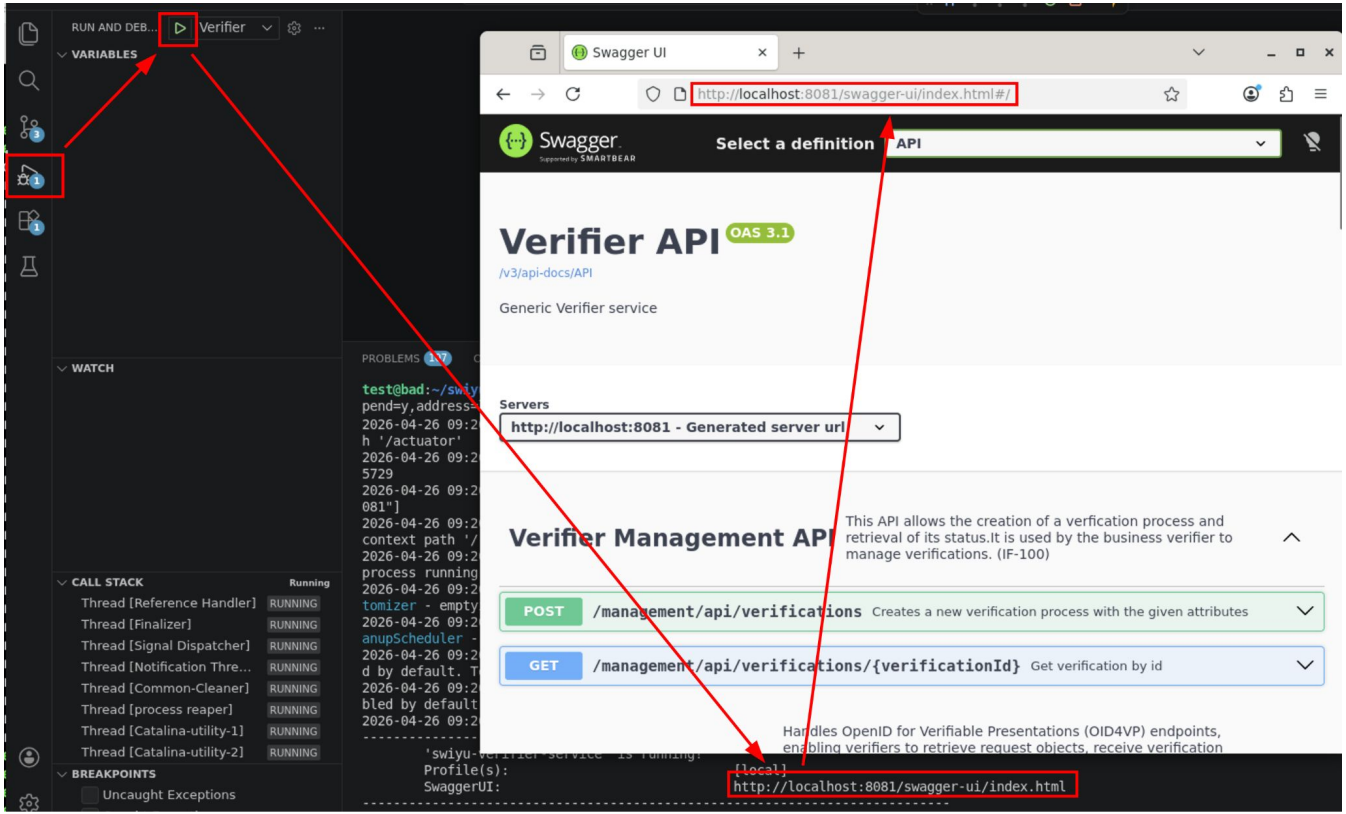
  external-url: "https://viscon.swiyu/verifier"

  client_id: "<go to your verifier_did.jsonl and paste here the value of
$.[3].value.id>"

spring:
  datasource:
    url: "jdbc:postgresql://localhost:5435/verifier_db"
```

Let's try it out!

~$ `code swiyu-verifier` (click "**Use Maven**" if prompted by the notification)



REDGUARD

Verifying our credential

My name is Bond
James Bond

REDGUARD
SECURING YOUR ASSETS

1. Create presentation request:

```
$ curl -ki https:// viscon.swiyu/verifier/management/api/verifications -H
'Content-Type: application/json' --data
'{"accepted_issuer_dids":[" <YOUR_ISSUER_DID>"],"response_mode":"direct_post","
presentation_definition":{"format":{"vc+sd-jwt":{"sd-jwt_alg_values":["ES256"
],"kb-jwt_alg_values":["ES256"]}},"input_descriptors":[{"id":"11111111-1111-1
111-1111-111111111111","format":{"vc+sd-jwt":{"sd-jwt_alg_values":["ES256"],"
kb-jwt_alg_values":["ES256"]}},"constraints":{"id":" visconid","name":"string",
"purpose":"string","fields": [{"path":["$.age_over_18"]}]}}]},"configuration_ov
erride":{}}'
```

```
{"id":"f14e<...>558","request_nonce":"zz<...>FD","state":"PENDING","presentat
ion_definition":{"format":{"vc+sd-jwt":{"sd-jwt_alg_values":["ES256"],"kb-jwt
_alg_values":["ES256"]}},"input_descriptors":[{"id":"111<...>11","format":{"v
c+sd-jwt":{"sd-jwt_alg_values":["ES256"],"kb-jwt_alg_values":["ES256"]}},"con
straints":{"id":"visconid","name":"string","purpose":"string","format":{},"fi
elds":[{"path":["$.age_over_18"]}]}}]},"verification_url":"https://viscon.swi
yu/verifier/oid4vp/api/request-object/f14e<...>558","verification_deeplink":"
swiyu-verify://?client_id=<verifier_did>&request_uri=https%3A%2F%2Fviscon.swiyu%
2Fverifier%2Foid4vp%2Fapi%2Frequest-object%2Ff14<...>558"}
```


REDGUARD

2. Use your swiyu wallet to present the credential

Tip: `adb shell am start -a android.intent.action.VIEW -d '"swiyu-verify://?client_id=..."' ch.admin.foitt.swiyu.abn`

Or convert the link to a QR code (`qrencode`, any online QR code generator, ...) and scan it with the swiyu App

If everything goes well, you have successfully verified your credential!

🥳🎉🥳

Have fun, and report your findings!

- Bug bounty program: https://www.bugbounty.ch/en/swiyu-2/

- To select your targets/testcases, you can take some inspiration from the existing issues/discussions (e.g. https://github.com/swiyu-admin-ch/swiyu-issuer/issues)

- **Always keep the threat model in mind**: swiyu-admin-ch.github.io/threat-model/

Thanks for your participation!

Questions and feedback are welcome :)

foomanroot@disroot.org

REDGUARD
SECURING YOUR ASSETS